ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #8

DatabaseCraft v1.0

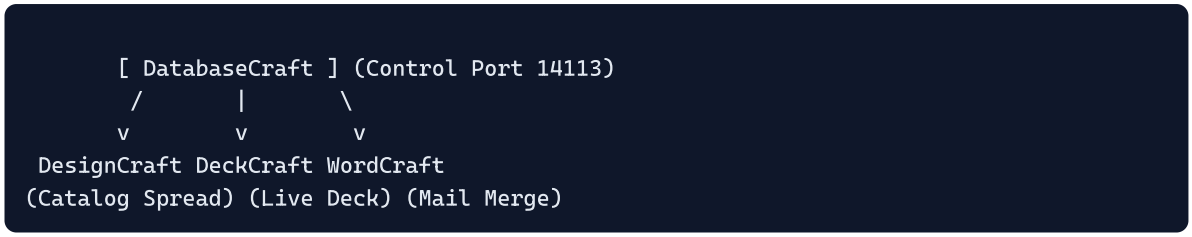
DatabaseCraft is the pure-Rust relational database, schema engine, and dynamic data-merge platform built for the Storytold ArtCraft creative ecosystem.



v1.0	4 Crates	8930	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\databasecraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

DatabaseCraft is the pure-Rust relational database, schema engine, and dynamic data-merge platform built for the **Storytold ArtCraft** creative ecosystem.



Features

- **Pure-Rust & Zero-Panic**: Enforces `deny(clippy::unwrap_used)` throughout.
- **Embedded & Local-First**: SQLite-backed `.dbcraft` storage with WAL mode, ACID transactions, and foreign key enforcement.
- **Universal Command Bus (`Command::dispatch`)**: CLI subcommands, MCP `tools/call`, and desktop UI execute the exact same typed commands.
- **Cross-Craft Data Merge**: Export query results directly formatted for:
- **DesignCraft**: Multi-page catalog and spread data merge.
- **DeckCraft**: Slide-ready KPI tables and charts.
- **WordCraft**: Mail-merge dictionaries and report variables.
- **GridCraft**: Seamless CSV/TSV table interchange.
- **PdfCraft**: Direct report and ledger layout tables.
- **Agent-Ready MCP Server**: Built-in line-delimited stdio Model Context Protocol (MCP `2024-11-05`) server.

Quickstart

```
# Show status and control port
cargo run -p databasecraft-mcp -- status

# Seed deterministic ecommerce demo data
cargo run -p databasecraft-mcp -- seed-demo --name ecommerce

# Query products table
cargo run -p databasecraft-mcp -- query --sql "SELECT sku, title, price FROM products;"

# Export formatted data merge for DesignCraft
cargo run -p databasecraft-mcp -- export-merge --query "SELECT sku, title, price,
hero_image FROM products;" --target designcraft

# Run Model Context Protocol (MCP) server
cargo run -p databasecraft-mcp -- mcp

# Run desktop app in headless smoke verification mode
cargo run -p databasecraft-app -- --smoke
```

Status — does today / next week / non-goals

Does today: core/store/mcp/app with workspace clippy denies; real CLI (status/query/schema/record/seed/embed/search); 17 test fns; docs (4); `status` reports its control port.

Next week: password-URL redaction test; binary-name decision (`dbcraft-cli.exe`); 3 PLAN recipes green + compat baseline.

Non-goals: backbone, not app — blueprints compile in AppCraft; Postgres/MySQL paths need dockerized CI or stay sqlite-labeled.

Lineage — the Storytold craft-app pattern

Storytold ArtCraft (storytold/artcraft — "the IDE for artists") was the starting point. The 16 craft apps in the Storytold stable each ship one binary that is both the human CLI and the agent MCP server (stdio), headless-first, with `run.json` + report artifacts. That modus operandi and harness is what this repo implements; the code here is independent, the pattern is shared.

Product Requirements & Architecture Specification

Status: Active Core Project (2026-11-01)

Parent Standard: `mcp-central-docs/standards/RUST_ARTCRAFT_STANDARD.md`

Registered Control Port: **14113** (`mcp-central-docs/operations/WEBAPP_PORTS.md`)

---

1. Vision & Executive Summary

DatabaseCraft is the pure-Rust relational database engine, schema designer, and dynamic data-merge platform built specifically for the **Storytold ArtCraft** creative desktop suite (PhotoCraft, VectorCraft, FilmCraft, DesignCraft, WordCraft, GridCraft, DeckCraft, CadCraft, FigCraft, LightCraft, PdfCraft, SoundCraft, EffectCraft).

While creative apps excel at content creation (raster, vector, video, audio, typesetting), real-world production demands a **relational, metadata, and dynamic publishing substrate**:

1. Catalogs, yearbooks, directories, and brochures created in **DesignCraft** require structured data-merge records.

2. Pitch decks, investor updates, and financial presentations in **DeckCraft** require live analytical queries.

3. Contracts, invoices, and mass communications in **WordCraft** require relational mail-merge entities.

4. Terabytes of raw footage, multi-cam takes, and audio stems in **FilmCraft** and **SoundCraft** require rich metadata logging, cue sheets, and semantic asset indexing.

5. AI agents need an ACID-compliant, typed relational engine rather than brittle raw spreadsheet cells in GridCraft.

DatabaseCraft provides this keystone in a single, zero-panic, pure-Rust workspace operating over the universal `Command::dispatch` bus.

---

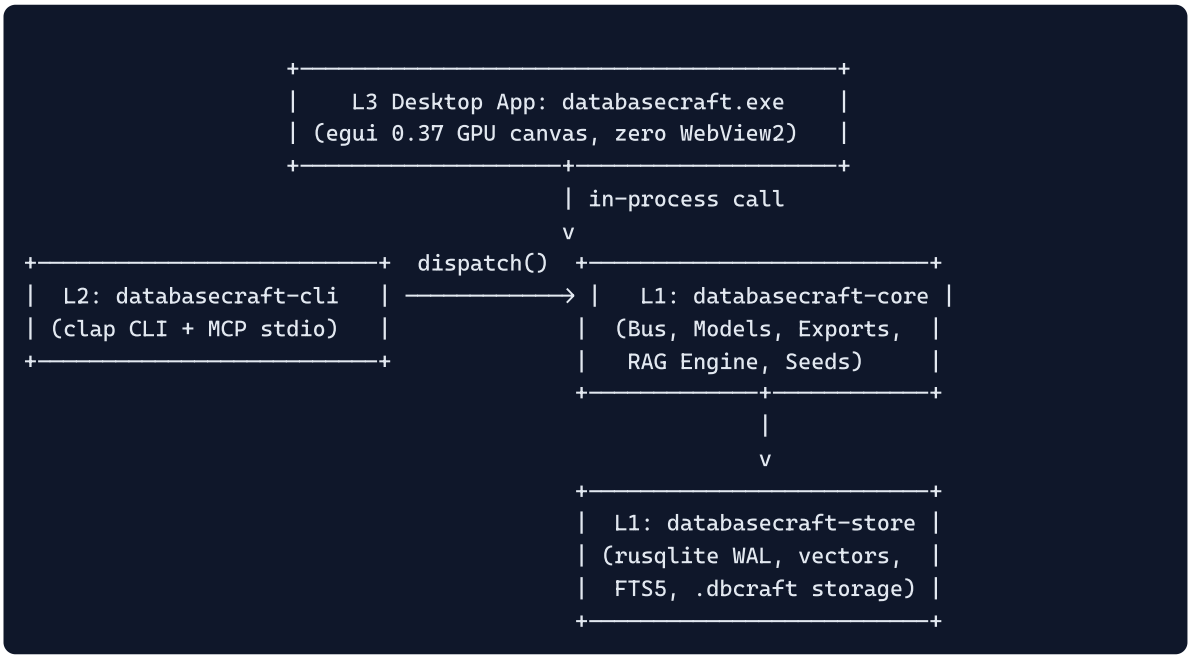
2. Target Users & Operating Personas

- **Creative Directors & Production Leads**: Managing shoot metadata, shot logs, call sheets, asset inventories, and multi-app publishing pipelines.
- **Autonomous AI Agents (Gemini, Claude Opus, Codex)**: Querying, validating, mutating, and feeding structured data across the 13 ArtCraft apps via line-delimited MCP stdio (`databasecraft-cli mcp`).
- **Technical Authors & Catalog Publishers**: Merging product databases directly into InDesign/DesignCraft spreads and WordCraft templates.
- **Data Analysts & Founders**: Assembling investor decks in DeckCraft backed by SQL / OLAP queries.

---

3. Five-Tier System Architecture

Following the **ArtCraft Containment Harness** (`CONTAINMENT_AND_CRATES.md`):



Layer Specifications

- **L1** `databasecraft-store`: Local-first relational storage wrapping `rusqlite` (WAL mode, foreign keys, busy timeout). Packed binary vector blobs, FTS5 full-text indexing, and zero-panic error handling.
- **L1** `databasecraft-core`: Schema DDL generator, validation fences (`validate_identifier`), cross-craft export formatting (DesignCraft, DeckCraft, WordCraft, GridCraft, PdfCraft), deterministic seeds (`ecommerce`, `film`, `kpi`), and the Universal Command Bus (`Command::dispatch`).
- **L2** `databasecraft-mcp`: Single binary providing human-friendly CLI subcommands (`--json`) and pure line-delimited JSON-RPC MCP `2024-11-05` server over stdio.
- **L3** `databasecraft-app`: Immediate-mode native desktop application calling `core` in-process with zero network sockets or child processes.

---

4. Functional Requirements

ID	Capability	Description	Status
F1	SQLite ACID Engine	Full transactional support, WAL journaling, foreign keys, and cascading constraints.	✅ Complete
F2	Universal Command Bus	100% of CLI, MCP, and UI actions dispatch through `Command::dispatch`.	✅ Complete
F3	Hostile Input Fences	Schema names and identifiers sanitized; zero direct indexing, zero unwrap.	✅ Complete
F4	Deterministic Oracles	Pre-canned test datasets (`ecommerce`, `film_production`, `executive_kpis`).	✅ Complete
F5	DesignCraft Data Merge	Formats SQL query rows into variable data spreads for page layouts.	✅ Complete
F6	DeckCraft Slide Export	Formats relational queries into slide tables and metric charts.	✅ Complete
F7	WordCraft Mail Merge	Emits structured mail-merge key/value records for documents.	✅ Complete
F8	GridCraft Interchange	Generates and parses RFC-4180 CSV strings for spreadsheet interop.	✅ Complete
F9	PdfCraft Ledger Export	Emits print-ready structured ledger table payloads.	✅ Complete
F10	Line-delimited MCP Stdio	Pure JSON-RPC on `stdout`, diagnostics strictly on `stderr`.	✅ Complete
F11	Dense Vector Math	Pure-Rust SIMD-friendly Cosine, DotProduct, and L2 distance on packed vectors.	✅ Complete
F12	SQLite FTS5 Indexing	Integrated full-text search with BM25 relevance ranking.	✅ Complete
F13	Hybrid RAG Facility	Combines dense vector search with sparse FTS5 using Reciprocal Rank Fusion (RRF).	✅ Complete
F14	Headless `--smoke` CI	Instantaneous executable verification without launching a GPU window.	✅ Complete
F15	Control Port `14113`	Registered live cross-craft orchestration channel.	✅ Complete
F16	5-View `egui` Desktop UI	Virtualized grid, form cards, ERD node canvas, SQL console, and media gallery.	✅ Complete
F17	Two-Plane Control Architecture	Live TCP listener on port `14113` (Plane 2) + command registry introspection (`commands`, `describe`).	✅ Complete
F18	In-Process Semantic Embeddings	Pure-Rust deterministic subword feature projection (`rag.embed_text`, `rag.semantic_search`).	✅ Complete
F19	Visual ERD & Live Grid Filter	Interactive ERD card canvas with FK links + real-time table grid filtering.	✅ Complete

---

5. UI/UX Requirements (Desktop App)

1. **Trade Dress**: Dark canvas (`#121316`), Slate-950 panels, hairline borders (`#26282E`), and Obsidian Emerald accents (`#108B7E`).

2. **Keyboard First**: Global palette (`Ctrl+K`), quick query run (`Ctrl+Enter`), cell navigation (`Arrows`), zoom (`Ctrl+Wheel`).

3. **5 Integrated Workspaces**:

- **Table Grid**: High-performance virtualized cell table with column sort, filter pills, and foreign-key link navigators.
- **Form Builder**: Access / FileMaker-style interactive cards for structured data entry.
- **Schema ERD Canvas**: Visual node graph connecting tables, primary keys, and foreign-key relations.
- **Query Console**: Syntax-highlighted SQL editor with results telemetry and export buttons.
- **Asset Gallery**: Thumbnail cards displaying linked creative assets (`.pcraft`, `.vcraft`, `.scraft`).

4. **Zero-Glitch Performance**: Immediate-mode GPU rendering at 60–144 FPS with sub-10ms UI latency.

---

6. Multi-App Production Flows

Registered in `mcp-central-docs/patterns/CRAFT_MULTI_APP_FLOWS.md`:

- **Flow 13: Product Catalog Factory**

`db:product-query (14113)` -> `photo:plate-grade (14109)` -> `design:print-spread (14102)` -> `pdf:portfolio (14108)`

- **Flow 14: Film Production Asset Logger**

`db:call-sheet-query (14113)` -> `sound:ident-jingle (14110)` -> `film:timeline-cut (14105)` -> `pdf:portfolio (14108)`

- **Flow 15: Executive Board Metric Deck**

`db:revenue-kpis (14113)` -> `grid:homepage-grid (14106)` -> `deck:pitch-deck (14101)` -> `pdf:portfolio (14108)`